

ARDUINO-BASED LINE FOLLOWING ROBOT *PROTOTYPE & SYSTEMS ENGINEERING*

PROJECT REPORT

Name	ID
Syed Jisan Ahmed	1233545
K M Mashrufur Rahman	1233549
Md Aynul Haque	1233987
Mohammad Taha Hossain Chowdhury	1234063

Course: **Prototype & System Engineering**
Institution: **Hochschule Hamm-Lippstadt (HSHL)**
Supervisor: **Dr. Faezeh Pasandideh & Fatema Idress**

1. Introduction & Objectives

Autonomous mechatronic systems are now central to modern industrial logistics — from warehouse AGVs to assembly-line inspection. This report documents the complete development of an Arduino-based line-following robot built for the Prototype & System Engineering course at HSHL. The project followed the full engineering lifecycle: requirements definition, CAD design, virtual simulation, physical assembly, embedded programming, and hardware debugging.

The system was built to meet four operational objectives:

- **Continuous Line Sensing:** detect a 22 mm black tape on white surface using dual TCRT5000 IR sensors.
- **Obstacle Avoidance:** detect objects within 25 cm using HC-SR04 ultrasonic sensors, stop safely, execute a geometric detour, and re-acquire the line.
- **State-Dependent Maneuvering (Bonus):** on detection of a second obstacle, execute a 180° in-place U-turn and reverse course.
- **Structural Integrity:** all components mounted using custom CAD-designed brackets. No glue, tape, or zip ties.

2. System Design & Engineering Approach

Rather than building by trial and error, the team used a structured Virtual-to-Physical (V2P) validation approach:

- **Virtual Prototype:** the complete circuit was modelled in Tinkercad. Push-button switches stood in for IR and ultrasonic sensors. This caught an IN3/IN4 wiring reversal before any hardware was connected.
- **Breadboard Integration:** a shared common ground was established across all modules. Without this, floating voltages caused random IR sensor outputs during initial testing.
- **Physical Assembly:** custom SolidWorks-designed holders were fabricated for motors, IR sensors, ultrasonic sensors, and battery/breadboard. This eliminated improvised mounting and ensured repeatable sensor geometry.

One significant design revision occurred during assembly: the original CAD placed IR sensors on top of the chassis, but wheel hardware blocked that position. A revised bracket was designed and fitted, mounting the sensors underneath at 5–6 mm ground clearance and 18–20 mm lateral spacing — within the optimal range for the TCRT5000 reading a 22 mm tape.

3. Hardware & Software Implementation

3.1 Hardware Architecture

Component	Model	Qty	Function
Microcontroller	Arduino Uno R3	1	Central processing unit
Motor Driver	L298N	1	PWM speed & direction control
DC Motors	TT Gear Motor	2	Differential drive
IR Sensors	TCRT5000	2	Line detection

Ultrasonic Sensors	HC-SR04	2	Obstacle distance measurement
Battery	9 V Alkaline	1	System power supply
Chassis	2WD Acrylic Kit	1	Mechanical platform

The ENA and ENB enable jumpers on the L298N were removed to allow the Arduino to send PWM signals through pins D5 and D6, enabling variable speed control via `analogWrite()`. With jumpers installed, motors run at fixed maximum power with no speed regulation.

3.2 Pin Configuration

Arduino Pin	Connected To	Direction	Function
D2, D3	IN1, IN2	OUT	Left motor direction
D4, D13	IN3, IN4	OUT	Right motor direction
D5 (PWM)	ENA	OUT	Left motor speed
D6 (PWM)	ENB	OUT	Right motor speed
D7, D8	Left/Right IR	IN	Line sensor readings
D9, D10	Left TRIG/ECHO	OUT/IN	Left ultrasonic
D11, D12	Right TRIG/ECHO	OUT/IN	Right ultrasonic
GND	Common rail	—	Shared ground for all modules

3.3 Software Logic

The software runs as a continuous polling loop with strict state-tracking variables managing all concurrent behaviour.

Line Following — Decision Table

Left IR	Right IR	Action	Meaning
BLACK	BLACK	<code>drive(75, 75)</code>	Centred — straight ahead
BLACK	WHITE	<code>drive(15, 80)</code>	Drifted right — correct left
WHITE	BLACK	<code>drive(80, 15)</code>	Drifted left — correct right
WHITE	WHITE	Pivot via <code>lastTurnDirection</code>	Line lost — recover from memory

When both sensors read WHITE simultaneously — common on tight curves — the `lastTurnDirection` variable (-1 left, 1 right) directs a pivot back toward where the line was last detected at `SEARCH_SPEED = 45 PWM`.

Distance Measurement: HC-SR04 distance is calculated as:

$$\text{distance (cm)} = \text{duration } (\mu\text{s}) \times 0.0343 \div 2$$

The pulseIn timeout was set to 30000 μs , covering the full usable range of the sensor (~514 cm). The original 12000 μs timeout caused readings to return 0 on some cycles, which were misinterpreted as no obstacle present — leading to missed detections.

Motor Reversal Flag: The right motor was physically mounted in the reverse orientation. Rather than rewiring it, a

RIGHT_MOTOR_REVERSED = true flag inside setMotor() inverts the speed value before setting direction pins. This corrects the physical reversal transparently at the driver level, so all higher-level steering commands use standard logical values.

4. Obstacle Avoidance & U-Turn Logic

4.1 Geometric Bypass Sequence

When either ultrasonic sensor reads below 25 cm, the avoidObstacle() function is triggered. The maneuver is a timed open-loop sequence:

Step	Action	Duration	Purpose
1	Hard brake	500 ms	Full stop — no coasting into box
2	Pivot right	340 ms @ speed 95	Swing nose clear of obstacle
3	Forward diagonal	650 ms @ speed 75	Clear obstacle side
4	Pivot left	340 ms @ speed 95	Re-align parallel to heading
5	Forward straight	1000 ms @ speed 75	Clear obstacle length
6	Pivot left inward	370 ms @ speed 95	Angle back toward line
7	Slow creep + IR scan	Up to 600 ms	Detect line at low speed
8	Turn 4 + extended crawl	Up to 5000 ms	Secondary search if missed

During Steps 7 and 8, the robot crawls at 35–45 PWM with IR sensors sampled every 2 ms. This sampling frequency ensures the 22 mm tape cannot pass beneath the sensors between cycles at the reduced speed. If the line is detected, a lock-on phase applies micro-corrections (nudge left or right) until both sensors confirm centre position.

4.2 180° U-Turn (Bonus Task)

An obstacleCount integer tracks cumulative detections. On the first obstacle, normal avoidance runs. On the second, uTurn180() is called:

```
void uTurn180() {
    brakeNow(); delay(300);
    drive(80, 0);          // single wheel pivot
    delay(1500);           // tune for exact 180° on surface
    brakeNow(); delay(200);
}
```

```
    obstacleCount = 0;    // reset for next lap
}
```

After the U-turn, normal line following resumes in the opposite direction along the same circular track. The 1500 ms duration was determined through physical calibration on the test surface.

5. Challenges & Solutions

Momentum Coasting: Setting motor speed to zero allowed 8–10 cm of post-stop coasting at operating speed. The robot made contact with the obstacle in early tests.

Solution: A `brakeNow()` function was implemented that sets all IN direction pins HIGH while applying maximum PWM simultaneously — creating an electrical short-circuit braking effect that locks the motors instantly.

Line Missed After Avoidance: After the bypass maneuver, the robot repeatedly crossed the 22 mm tape without the IR sensors registering it. At speeds above ~20 PWM, sampling was too infrequent relative to travel speed.

Solution: Creep speed reduced to 35–45 PWM. Sensors sampled every 2 ms inside a tight while loop. A secondary four-turn search sequence was added as a fallback if the primary search times out.

360° Spinning Instead of Pivoting: Early pivot commands using negative speed values caused continuous spinning. The `RIGHT_MOTOR_REVERSED` flag was double-inverting the sign — both wheels ended up rotating the same direction.

Solution: Pivot commands changed to `drive(SHARP_TURN_SPEED, -SHARP_TURN_SPEED)`. After the internal flag applies one inversion to the right motor, the result correctly produces opposing wheel directions.

IR Sensor Placement: The original CAD mounted sensors on top of the chassis. Wheel hardware blocked this position during assembly. Sensors needed to be at 5–6 mm ground clearance to detect the tape reliably.

Solution: A revised bracket was designed and fitted during the build, repositioning sensors underneath the chassis. Height and lateral spacing (18–20 mm) were measured and verified before permanent installation.

Dropped Ultrasonic Readings: The original pulseIn timeout of 12000 μ s caused the function to return 0 on some cycles — treated as 999 cm by the code. Genuine obstacle detections were silently dropped.

Solution: Timeout increased to 30000 μ s, covering the full HC-SR04 detection range. All readings within sensor specification are now captured reliably.

6. Conclusions & Future Improvements

Team E2 designed, simulated, assembled, and programmed a fully functional autonomous robot that reliably follows a line, avoids a physical obstacle, and executes a 180° directional reversal on the second obstacle encounter. The project was completed through a systematic engineering approach — virtual validation before hardware contact, structured debugging, and iterative physical tuning.

The most important lessons came from hardware — not simulation. The ENA/ENB physical reversal, the double sign-flip in motor logic, the dropped ultrasonic readings, and the IR sensor placement revision were all problems that only appeared when the robot was running on the actual track. Each one was diagnosed methodically and resolved without redesigning the broader system.

Future Improvements

- **PID Line Control:** replace discrete bang-bang turning with a Proportional-Integral-Derivative controller to eliminate micro-oscillations and produce smoother tracking on straight sections.
- **Rechargeable Power:** the 9 V alkaline battery suffers voltage sag as it drains, subtly altering motor speed and affecting the accuracy of timed maneuver steps. A Li-ion pack with a step-down voltage regulator would deliver consistent power across the full operational period.
- **Bluetooth Override:** an HC-05 module would allow real-time parameter adjustment and manual override without re-uploading code.
- **Maze-Solving Extension:** the existing obstacle avoidance logic could be extended into a left-hand rule or flood-fill maze-solving algorithm for more complex navigation scenarios.

References

- [1] Arduino LLC. Arduino Uno R3 Official Documentation. <https://docs.arduino.cc>
 - [2] STMicroelectronics. L298N Dual Full-Bridge Driver Datasheet.
 - [3] Vishay Semiconductors. TCRT5000 Reflective Optical Sensor Datasheet.
 - [4] HC-SR04 Ultrasonic Ranging Module — Technical Specification. Cytron Technologies.
 - [5] Autodesk Inc. Tinkercad Circuit Simulator. <https://www.tinkercad.com>
 - [6] Dassault Systèmes. SolidWorks CAD Software. <https://www.solidworks.com>
 - [7] Course Materials — Prototype & System Engineering, HSHL, 2025/2026.
-